

도커 실습

국립목포대학교
컴퓨터공학과

이영호 교수

Docker 실습 과정

초보자를 대상으로 Docker의 기본 실습 과정을 안내합니다. 이 과정에서는 Dockerfile을 생성하고 이미지를 만들고 실행하며, `commit` 명령어를 사용하여 자신만의 Docker 이미지를 만드는 방법을 다룹니다.

1. Docker란?

Docker는 애플리케이션을 컨테이너 안에서 실행할 수 있게 도와주는 도구입니다. 컨테이너는 애플리케이션과 그에 필요한 모든 라이브러리, 종속성을 하나의 패키지로 만들어 어디서나 일관되게 실행할 수 있도록 합니다.

2. 준비 사항

- Docker가 설치된 시스템 (Linux, macOS, Windows)
- 터미널 또는 명령 프롬프트

3. 앱 파일 작성하기

1. `app.py` 작성

간단한 Flask 웹 서버를 만드는 `app.py` 파일을 생성합니다.

```
# app.py
from flask import Flask
app = Flask(__name__)
@app.route('/')
def hello():
    return "Hello, Docker!"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

2. `requirements.txt` 작성

Flask 라이브러리를 명시한 `requirements.txt` 파일을 생성합니다.

```
flask==2.3.2
```

이 파일은 Flask를 설치하기 위해 필요합니다.

4. Dockerfile 작성하기

Dockerfile은 Docker 이미지를 생성할 때 사용되는 텍스트 파일입니다. 이 파일에는 이미지 빌드 과정에서 실행될 명령어들이 정의됩니다.

1. **디렉토리 생성:** 먼저 새로운 디렉토리를 만들고, 그 안에서 Dockerfile을 작성합니다.

```
mkdir my-docker-app  
cd my-docker-app
```

2. **Dockerfile 생성**: 해당 디렉토리에서 **Dockerfile** 이라는 파일을 생성하고, 아래 내용을 추가합니다.

```
FROM python:3.9
WORKDIR /app
COPY . /app
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

이 Dockerfile은 Python 3.9를 기반으로 하며, 작업 디렉토리를 **/app**으로 설정합니다. **requirements.txt** 파일이 있다면 필요한 패키지를 설치하고, **app.py** 파일을 실행합니다.

5. Docker 이미지 생성하기

Dockerfile과 앱 파일을 이용해 Docker 이미지를 생성합니다.

1. **이미지 빌드**: 아래 명령어를 사용하여 이미지를 빌드합니다.

```
docker build -t my-python-app .
```

`my-python-app`은 생성될 이미지의 이름이며, `.`은 현재 디렉토리를 의미합니다.

2. **이미지 확인:** 빌드가 완료되면, 이미지 목록을 확인할 수 있습니다.

```
docker images
```

여기서 `my-python-app`이라는 이미지가 생성된 것을 볼 수 있습니다.

6. Docker 컨테이너 실행하기

이미지를 이용해 컨테이너를 실행합니다.

1. 컨테이너 실행: 아래 명령어로 컨테이너를 실행합니다.

```
docker run -d -p 5000:5000 my-python-app
```

- `-d`: 백그라운드 모드로 실행합니다.
- `-p 5000:5000`: 로컬의 5000번 포트를 컨테이너의 5000번 포트에 연결합니다.
- `my-python-app`: 실행할 이미지 이름입니다.

2. 브라우저에서 확인: 브라우저에서 `http://localhost:5000` 을 열면 "Hello, Docker!" 메시지를 확인할 수 있습니다.

7. 변경 사항 저장하기 (commit)

컨테이너에서 변경한 내용을 저장하여 새로운 이미지를 생성할 수 있습니다.

1. **컨테이너에 접속:** 실행 중인 컨테이너에 접속합니다.

```
docker exec -it [컨테이너 ID] /bin/bash
```

2. **변경 작업 수행:** 컨테이너 내부에서 파일을 수정하거나 패키지를 설치하는 등의 작업을 수행합니다.

- app.py를 수정하여 여러분의 개인 홈페이지를 만들어 봅니다.

3. 변경 사항 저장: 변경한 내용을 새로운 이미지로 저장합니다.

```
docker commit [컨테이너 ID] my-python-app-v2
```

- [컨테이너 ID] 는 `docker ps` 명령어로 확인할 수 있는 실행 중인 컨테이너의 ID입니다.
- `my-python-app-v2` 는 새롭게 생성할 이미지의 이름입니다.

4. 새로운 이미지 확인: 이미지 목록을 다시 확인하면, `my-python-app-v2` 가 추가된 것을 볼 수 있습니다.

```
docker images
```

8. 정리

- Dockerfile을 통해 이미지 생성 및 컨테이너 실행이 가능하며, `commit` 명령어로 컨테이너의 변경 사항을 저장할 수 있습니다.
- Docker를 사용하면 애플리케이션을 일관되게 배포하고 관리할 수 있어 매우 유용합니다.
- 이 과정을 통해 `Docker`의 기본적인 사용 방법을 이해하고, 직접 이미지를 생성하고 커스터마이징하는 방법을 익힐 수 있습니다. 추가적으로 Docker의 다양한 기능을 학습하며 활용할 수 있습니다!